

ADRDS: Adaptive Dynamic Ransomware Detection System

S. Anupama Kumar^{1*}, Manish Tandurkar², Manish H Parashar²,
Nikhil N Bharadwaj², Vinay Sinnur³, Aditya Tripathi¹

^{1*}Dept. of Artificial Intelligence and Machine Learning, RV College of
Engineering, Bangalore, India.

²Dept. of Computer Science, RV College of Engineering, Bangalore,
India.

³Dept. of Information Science, RV College of Engineering, Bangalore,
India.

*Corresponding author(s). E-mail(s): anupamakumar@rvce.edu.in;
Contributing authors: manishtandurkar.cs23@rvce.edu.in;
manishhparashar.cs23@rvce.edu.in; nikhilnb.cs23@rvce.edu.in;
vinaysinnur.is23@rvce.edu.in; adityatripathi.ai23@rvce.edu.in;

Abstract

Ransomware has emerged as a critical threat to organizational and individual security, characterized by file encryption, extortion, and substantial operational disruption. Traditional signature-based detection mechanisms exhibit limited efficacy against novel and obfuscated variants, particularly zero-day exploits that evade static pattern-matching approaches.

This paper presents ADRDS (Adaptive Dynamic Ransomware Detection and Response System), an AI-driven platform employing behavior-based analysis to identify ransomware activity. The system integrates a secure behavioral simulator that replicates ransomware execution patterns—including entropy anomalies, atypical API invocation sequences, and elevated file I/O bursts—without deploying malicious binaries. Behavioral telemetry is processed through a hybrid machine learning architecture comprising supervised classifiers trained on established datasets such as BigMalware and EMBER, alongside an unsupervised LSTM-based autoencoder designed to detect previously unseen variants.

Detection outputs are fused into a unified risk metric, enabling early-stage threat identification with enhanced confidence. The platform incorporates automated response simulations and a real-time visualization dashboard, facilitating transparent, step-by-step observation of detection processes. Experimental validation

demonstrates that the proposed hybrid strategy achieves robust detection performance while maintaining operational safety and scalability. ADRDS serves as a valuable resource for security research, interactive training environments, and the development of advanced cyber defense systems.

Keywords: Ransomware detection, Behavioral analysis, Machine learning, Security demonstration, Threat modeling, Intrusion detection

1 Introduction

Ransomware has rapidly evolved into one of the most disruptive threats in digital security. Hospitals, educational institutions, critical infrastructure, and businesses have experienced significant operational paralysis due to encryption-based extortion. The threat landscape has shifted significantly; many strains now exfiltrate sensitive data prior to encryption, leveraging double extortion tactics [?]. Furthermore, ransomware developers continuously modify code to evade traditional signature-based antivirus solutions [?]. Consequently, static defenses often fail to protect against these dynamic and polymorphic threats.

The proliferation of ransomware attacks presents a multifaceted challenge that extends beyond technical considerations. According to recent industry reports, global ransomware damages are projected to exceed \$265 billion annually by 2031, with attacks occurring approximately every two seconds [?]. The financial impact encompasses not only ransom payments but also operational downtime, data recovery costs, regulatory penalties, and reputational damage. Healthcare organizations face particularly severe consequences, as ransomware incidents have been directly linked to treatment delays and patient safety concerns.

The sophistication of modern ransomware has increased substantially. Ransomware-as-a-Service (RaaS) platforms have democratized access to advanced attack capabilities, enabling less technically proficient adversaries to deploy sophisticated encryption schemes. Furthermore, the emergence of fileless ransomware, which operates entirely in memory without writing malicious executables to disk, circumvents traditional file-based detection mechanisms. These developments necessitate a fundamental shift from reactive signature-matching to proactive behavioral analysis.

A critical challenge in ransomware defense is the *detection lag*—the interval between the execution of malicious code and the response of a security mechanism. During this window, ransomware can compromise thousands of files, often causing irreversible damage despite backup strategies. Reports like the Verizon Data Breach Investigations Report [?] indicate that while detection rates have improved, the velocity and precision of response mechanisms require further optimization.

The asymmetry between attacker and defender capabilities compounds this challenge. Attackers require only a single successful intrusion to cause significant damage, whereas defenders must maintain comprehensive protection across all potential attack surfaces. This asymmetry is particularly pronounced in organizational environments

where legacy systems, diverse software ecosystems, and varying security configurations create heterogeneous threat landscapes.

Another significant limitation is the “black box” nature of many machine learning-based detection tools. While accurate, these systems often lack interpretability. Security analysts require granular insights—such as the identification of unusual API calls, encrypted file write spikes, or tampering with recovery options—to validate alerts. Without such context, reliance on automated systems remains precarious in high-stakes environments. The opacity of detection decisions impedes incident response workflows and complicates forensic analysis.

Additionally, the persistence of false positives presents operational challenges. Security teams overwhelmed by spurious alerts may develop alert fatigue, potentially overlooking genuine threats. Balancing sensitivity with specificity requires careful threshold calibration and continuous model refinement based on environmental feedback.

To address these limitations, this paper introduces ADRDS, the AI-Driven Ransomware Detection and Response Simulator. ADRDS is an interactive, behavior-centric platform designed to bridge the gap between theoretical machine learning research and practical security operations. In contrast to signature-based approaches, this system analyzes software execution behavior, including file interactions, process generation, and system modifications.

The system operates within a secure, sandboxed environment. Users can initiate simulated ransomware activities to observe the real-time scoring of risks, the identification of attack stages, and the generation of telemetry. This feedback loop concretizes abstract detection concepts, rendering ADRDS applicable for both advanced research and educational purposes.

ADRDS distinguishes itself through three core design principles. First, *transparency*: every detection decision is traceable to specific behavioral indicators, enabling users to understand why particular activities trigger alerts. Second, *modularity*: the architecture accommodates component replacement and extension without requiring comprehensive system redesign. Third, *safety*: all experimentation occurs within isolated environments using simulated behaviors, eliminating the risks associated with live malware execution.

1.1 Contributions

This work makes several key contributions to the field. First, we define a behavioral taxonomy that maps observable host-level events—such as file access bursts and process anomalies—to specific stages of the ransomware attack lifecycle. This taxonomy provides a structured framework for feature extraction and detection logic. Second, we present the ADRDS architecture, a comprehensive system integrating behavioral simulation, hybrid machine learning detection, and real-time visualization. The architecture enables controlled experimentation while maintaining operational safety. Third, we provide experimental validation of detection accuracy and response latency, demonstrating that the proposed approach detects threats early without degrading system performance. Fourth, we establish a reproducible research platform, offering a modular and extensible simulator suitable for adaptation by educators and researchers. The

platform facilitates curriculum development, hypothesis testing, and the evaluation of novel detection methodologies.

1.2 Organisation of Paper

The remainder of this paper is organized as follows. Section II presents a comprehensive review of related work in ransomware detection, highlighting the evolution of detection methodologies and identifying gaps in existing approaches. Section III establishes the threat model and discusses safety considerations essential for conducting ransomware research in controlled environments. Section IV provides a detailed system overview of the ADRDS architecture, describing the interaction between its core components. Section V introduces the behavioral taxonomy and simulation model that underpin our detection framework, mapping observable system events to ransomware attack stages. Section VI details the implementation specifics, including technology stack choices, algorithmic implementations, and system integration strategies. Section VII presents demonstration scenarios and provides a walkthrough of the system’s operation in realistic attack contexts. Section VIII describes the experimental setup, including dataset selection, evaluation metrics, and testing environments. Section IX presents comprehensive results and evaluation, demonstrating the effectiveness of our approach across multiple performance dimensions. Section X discusses the implications of our findings, acknowledges system limitations, and identifies areas for improvement. Finally, Section XI concludes the paper and outlines directions for future research in behavioral ransomware detection.

2 Background and Related Work

Ransomware detection remains a critical area of research, driven by the continuous evolution of extortion tactics. The security community has explored a spectrum of methodologies, ranging from signature-based matching to advanced machine learning models [? ?]. This section reviews principal approaches and situates ADRDS within the existing body of work.

2.1 Evolution of Detection Techniques

Traditional antivirus solutions employing signature-based methods rely on maintaining databases of known malware signatures. While effective against established threats, this paradigm is insufficient against polymorphic ransomware that dynamically alters its code, or zero-day variants lacking prior cataloging [?]. Consequently, signature-based reliance is increasingly untenable.

Behavior-based detection represents a significant advancement, monitoring execution patterns rather than static code attributes. By tracking file system activity, registry modifications, process behaviors, and system call sequences, detection systems can identify emerging attacks. Pioneering work by Continella et al. [?] and Scaife et al. [?] demonstrated that monitoring abnormal file access and encryption rates facilitates early detection. Chen and Bridges [?] further developed automated behavioral pattern extraction techniques for early-stage identification. The RWGuard system

[?] introduced real-time detection mechanisms specifically targeting cryptographic ransomware. However, distinguishing malicious activity from legitimate operations performed by backup utilities or compression tools remains a challenge, often leading to false positives.

Machine learning approaches have emerged to enhance detection precision [?]. Supervised classifiers, such as Random Forests, Support Vector Machines, and gradient-boosted trees trained on labeled datasets, can identify subtle patterns compliant with known attack vectors. Deep learning models, including recurrent neural networks and autoencoders, have also shown efficacy in capturing temporal behaviors and anomalies [?]. Studies by Sgandurra et al. [?] and Vinayakumar et al. [?] report promising results. Mirsky et al. [?] demonstrated the effectiveness of autoencoder ensembles for network intrusion detection, inspiring similar approaches for host-based detection. However, many existing systems operate offline, exhibit high latency, or function as opaque models lacking interpretability.

Hybrid systems integrate multiple methodologies, combining static and dynamic analysis alongside supervised and unsupervised learning to improve resilience [?]. Research by Kolosnjaji et al. [?] and Pascanu et al. [?] indicates that hybrid models offer superior generalization and noise tolerance. The DREBIN system [?] demonstrated that explainable detection models can achieve both high accuracy and interpretability. Nevertheless, these systems often incur significant computational overhead and typically lack interactive capabilities suitable for educational or exploratory use.

Complementary to host-based detection, network-based mitigation approaches have emerged for ransomware mitigation. Cabaj and Mazurczyk [?] proposed using software-defined networking to detect and block ransomware communication patterns, particularly for variants like CryptoWall that rely on command-and-control infrastructure.

Visualization and analysis techniques have also proven valuable for malware classification. Nataraj et al. [?] pioneered the visualization of malware as images for classification, while Huang and Stokes [?] extended these techniques to visualized image matrices for detailed malware analysis. These approaches provide intuitive representations that complement traditional feature-based detection.

2.2 Adversarial Considerations

As machine learning-based detection has matured, adversarial techniques have emerged to evade these systems. Maiorca et al. [?] provide comprehensive analysis of adversarial approaches in malware detection, highlighting the importance of robust feature selection and model design. Xu et al. [?] address these concerns through multi-layer detection architectures that resist evasion attempts.

2.3 Interactive and Demonstration Platforms

Despite advances in detection accuracy, limited tools prioritize hands-on exploration. Most platforms operate as automated backend services with minimal visibility into decision-making processes. Existing educational sandboxes, such as CyberSecLab

[?] , enable experimentation with general security concepts but often lack specialized ransomware behavioral taxonomies, real-time machine learning integration, and transparent logic visualization.

2.4 System Positioning

ADRDS addresses this gap by providing an interactive environment. Unlike “black-box” systems, ADRDS enables users to trigger simulated ransomware behaviors, adjust parameters, and observe the detection engine’s response in real time. The system combines safe behavioral simulation with a hybrid ML detector and an interactive dashboard. This approach provides both rigorous detection capabilities and the necessary transparency to understand alert triggers, making ADRDS valuable for advanced security research and instructional applications.

2.5 Foundational Research

The architecture and evaluation of ADRDS are informed by several key studies. Kharraz et al. [?] provide analysis of the fundamental mechanisms of ransomware attacks. Continella et al. [?] propose ShieldFS, a self-healing filesystem that influenced the behavioral monitoring approach used here. Scaife et al. [?] introduce CryptoLock, focusing on preventing data loss. Sgandurra et al. [?] provide a framework for automated dynamic analysis. Vinayakumar et al. [?] survey deep learning approaches, while Kolosnjaji et al. [?] explore hybrid analysis using system call sequences. Pascanu et al. [?] demonstrate the utility of Recurrent Neural Networks (RNNs) in malware classification, and Lee et al. [?] present CyberSecLab, which inspired the educational and interactive focus of this work.

3 Threat Model and Safety Considerations

This section outlines the assumptions regarding the adversary, the scope of targeted threats, and the safety protocols implemented to ensure secure experimentation.

3.1 Adversarial Assumptions

We focus on ransomware that has successfully established a presence on a host system. Specifically, the adversary is assumed to possess the capability to execute malicious code following an initial compromise, invoke standard system APIs and cryptographic libraries for file encryption, and perform high-volume file operations such as reads, writes, and renames with anomalous frequency. Additionally, the attacker may spawn new processes, attempt privilege escalation, establish persistence mechanisms, and optionally communicate with a command-and-control server for key exchange or status reporting.

The adversary model encompasses several ransomware categories based on their operational characteristics. *Crypto-ransomware* focuses primarily on file encryption, employing hybrid cryptographic schemes combining symmetric and asymmetric algorithms to ensure data remains inaccessible without payment. *Locker ransomware*

restricts system access without necessarily encrypting files, typically targeting system boot processes or user interface components. *Wiper variants* masquerade as ransomware but prioritize data destruction over extortion, often employed in state-sponsored or destructive attacks. ADRDS is primarily designed to detect crypto-ransomware due to its prevalence and the distinctive behavioral signatures associated with encryption operations.

The ransomware attack lifecycle comprises distinct phases, each exhibiting characteristic behavioral patterns. Understanding these phases informs the detection strategy and enables stage-aware alerting.

The attack typically begins with initial access and execution, during which, following successful infection through vectors such as phishing, exploit kits, or supply chain compromise, the ransomware payload executes on the target system. This phase may involve unpacking, decryption of embedded components, and initial environment reconnaissance. Subsequently, attackers frequently attempt privilege escalation to gain broader system access, which may involve exploiting local vulnerabilities, abusing misconfigured services, or leveraging credential theft techniques. Elevated privileges enable encryption of protected system files and facilitate persistence.

To survive system reboots and ensure continued access, ransomware often establishes persistence through registry modifications, scheduled tasks, or service installation. These activities generate observable artifacts that contribute to detection. In enterprise environments, ransomware may propagate to additional systems through lateral movement via network shares, remote execution protocols, or exploitation of networked services, amplifying attack impact and complicating remediation.

Modern ransomware increasingly incorporates data exfiltration prior to encryption, enabling double extortion tactics through file identification, compression, and transmission to attacker-controlled infrastructure. The final operational phase involves encryption and ransom demand, characterized by systematic encryption of identified files and presentation of ransom demands. This phase exhibits the most distinctive behavioral signatures, including high-volume file operations and entropy anomalies.

From a defensive perspective, it is assumed that the monitoring system maintains visibility over host-level telemetry, including file system events, process activity, API calls, and resource utilization. ADRDS specifically targets the pre-encryption and active encryption phases, as these periods exhibit behavioral divergence from standard operational workloads. Certain vectors are explicitly excluded from the scope of this study, including initial infection vectors such as phishing campaigns, exploit kits, or supply-chain compromises. Furthermore, post-encryption activities, including ransom negotiation, payment infrastructure, and monetization strategies, are not addressed.

3.2 Safety Protocols

ADRDS avoids the execution of live ransomware binaries, instead utilizing parameterized behavioral simulations to replicate indicators of compromise, such as entropy spikes, rapid file writes, and anomalous process activity, without executing actual encryption or destructive payloads. This simulation-based validation design ensures that demonstrations and experiments remain inherently safe. The simulation engine implements precise control over behavioral parameters, enabling the reproduction

of specific ransomware families and attack scenarios. Parameters include encryption velocity, file targeting patterns, process spawning behavior, and resource utilization profiles, supporting both research requiring specific attack characteristics and educational demonstrations requiring progressive threat escalation.

All experiments are conducted within sandboxed, air-gapped environments to ensure environment isolation, devoid of connectivity to production systems or sensitive networks. This isolation mitigates the risk of accidental data exposure and prevents lateral movement of simulated threats. Network interfaces are disabled or restricted to local communication, and file system access is confined to designated experimental directories.

Regarding ethical considerations, ADRDS is developed strictly for defensive research, education, and demonstration purposes. The system does not contain exploit code or instructional material susceptible to weaponization. Users are expected to adhere to applicable legal frameworks, institutional policies, and ethical guidelines when utilizing the platform. The development team maintains responsible disclosure practices and engages with the security research community to ensure appropriate use.

3.3 Privacy and Data Handling

The system does not collect, store, or transmit personally identifiable information (PII). All telemetry utilized for detection and visualization is either synthetically generated or derived from non-sensitive experimental datasets, ensuring alignment with privacy-preserving research standards. Data retention policies limit storage duration, and all experimental data remains local to the execution environment.

4 System Overview

ADRDS is designed as a modular, interactive system bridging abstract detection research with empirical experimentation. As illustrated in Fig. ??, the architecture comprises three integrated components: a behavioral simulator, a machine learning detection engine, and an interactive visualization layer. These components facilitate the simulation of ransomware activity, real-time detection monitoring, and the exploration of parameter sensitivity.

4.1 System Architecture

4.1.1 Behavioral Simulator

The behavioral simulator functions as the core of ADRDS, generating realistic ransomware behavior sequences without the execution of live malware. It implements a configurable state machine modeling the attack lifecycle—including reconnaissance, privilege escalation, persistence, encryption, and optional command-and-control communication. Each event corresponds to observable host-level activity, such as file operations, process creation, and resource utilization spikes. Parameters including encryption rate, file targeting strategy, and execution intensity are adjustable, allowing for the emulation of diverse ransomware families and tactics.

4.1.2 Machine Learning Detector

The detection engine processes structured features extracted from simulator output, applying trained machine learning models to classify activity as benign or malicious. The system supports multiple classifiers, including Random Forests [?], Support Vector Machines [?], and neural networks, enabling performance comparison under varying conditions. A critical feature is the adjustable confidence threshold, which allows users to modulate detection sensitivity and analyze the trade-off between false positive rates and detection capabilities.

4.1.3 Interactive Visualization Layer

The visualization layer transforms raw telemetry into interpretable insights. It renders attack timelines, feature histograms, and risk indicators in real time. The dashboard interface permits the modification of simulation settings and detection thresholds, reflecting the immediate impact of these adjustments without requiring experiment restarts.

4.2 Data Flow

ADRDS operates via a closed-loop pipeline ensuring synchronization between simulation, detection, and visualization. The process initiates with the behavioral simulator generating timestamped event streams corresponding to each attack stage. Subsequently, a feature extraction module converts these raw events into structured feature vectors, aggregating stage context with behavioral metrics. The machine learning detector then evaluates these vectors, producing classification outputs and confidence scores. Finally, the visualization layer renders the results, attack progress, and performance metrics in real time. User interactions—such as parameter adjustments and threshold modifications—are fed back into the simulator, closing the control loop.

This architecture establishes ADRDS as a dual-purpose platform, suitable for both demonstration and research. The modular design ensures that individual components can be replaced or extended, such as incorporating new models or feature extractors, without disrupting the overall pipeline.

5 Behavioral Taxonomy

Catching ransomware early requires understanding how it behaves over time. Rather than hunting for static signatures, we structure detection around a taxonomy of observable behaviors that unfold across distinct attack stages. This section lays out that taxonomy and explains how it feeds into feature extraction and detection logic.

5.1 Stages of a Ransomware Attack

We model ransomware as a sequence of stages, each defined by the host-level events it produces. Recognizing these stages helps ADRDS detect partial attacks and estimate how far an intrusion has progressed.

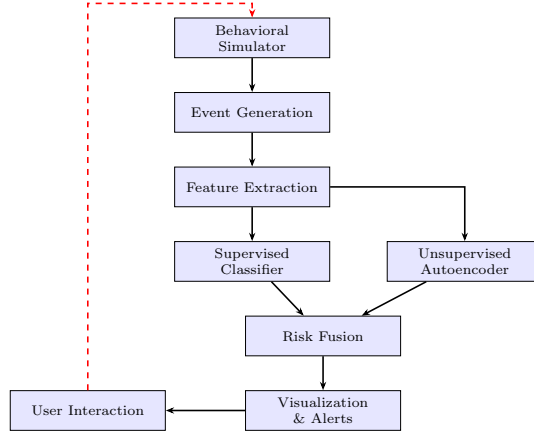


Fig. 4.1 ADRDS system architecture illustrating data flow between the behavioral simulator, machine learning-based detector, and interactive visualization components. The dashed line represents the feedback loop.

5.1.1 Stage 1: Reconnaissance and Setup

In the opening moments, ransomware probes its environment. It might enumerate directories, query system information, or check for virtual machine artifacts that could indicate a sandbox. These actions are subtle, but they set the stage for what comes next. Observable indicators include elevated directory listing operations, registry queries for system configuration, and process enumeration activities. The reconnaissance phase typically exhibits lower activity intensity compared to subsequent stages, making early detection challenging but not impossible.

5.1.2 Stage 2: Privilege Escalation and Persistence

Once reconnaissance wraps up, attackers often try to strengthen their grip. We see abnormal process spawning, modifications to startup entries, and interactions with system-level resources. These moves expand access and help the malware survive reboots. Specific indicators include modifications to autorun registry keys, creation of scheduled tasks, service installation attempts, and interactions with Windows Management Instrumentation (WMI). The temporal clustering of these activities distinguishes malicious behavior from legitimate administrative operations.

5.1.3 Stage 3: File Enumeration and Targeting

Before encrypting anything, ransomware identifies its targets. This stage features high-volume read operations, rapid directory traversal, and selective filtering based on file extensions or folder paths. The burst of file activity stands out sharply against typical user workloads. Common targeting strategies include extension-based filtering (prioritizing documents, databases, and media files), path-based exclusion (avoiding system directories to maintain operability), and size-based selection (targeting files within specific size ranges to optimize encryption throughput).

5.1.4 Stage 4: Encryption and Destruction

Here the damage happens. Intensive file writes, sudden renames, and sharp entropy spikes mark the encryption phase. Files transform from readable documents into scrambled blobs at a pace that no ordinary application would match. The encryption phase exhibits distinctive characteristics including sustained high write throughput, dramatic entropy increases in modified files, systematic file renaming with appended extensions, and deletion or modification of Volume Shadow Copies. The intensity and patterns of these operations provide strong detection signals.

5.1.5 Stage 5: Communication and Wrap-Up

Some ransomware phones home after encryption—exchanging keys, reporting success, or awaiting further instructions. We watch for anomalous network connections and communication patterns that coincide with the encryption window. Indicators include connections to previously unobserved external hosts, encrypted traffic to non-standard ports, and communication periodicity aligned with attack progression.

5.2 Observable Indicators and Feature Extraction

Each stage translates into measurable indicators that ADRDS extracts and monitors. The feature extraction process transforms raw telemetry into structured representations suitable for machine learning analysis.

File system metrics constitute a primary data source, encompassing access frequency across file types, modification rate relative to baseline patterns, entropy changes in modified content, directory traversal depth and breadth, and file operation sequencing patterns. Entropy analysis proves particularly valuable, as legitimate file modifications typically produce modest entropy changes, whereas encryption dramatically increases file entropy toward maximum theoretical values. Process-level attributes provide complementary insight through monitoring of CPU utilization patterns, memory consumption trajectories, child process spawning rates, API call frequencies and sequences, and handle creation for files and system objects. Ransomware typically exhibits elevated resource consumption during encryption phases, providing additional detection opportunities.

Temporal patterns further enhance detection capability, incorporating burstiness metrics that quantify activity concentration, sustained high-activity window detection, inter-event timing analysis, and phase transition identification. The temporal dimension enables differentiation between continuous legitimate workloads and the episodic intensity characteristic of ransomware operations. Each observation is additionally annotated with stage context indicating the most probable attack stage based on behavioral patterns, enabling stage-aware alerting and response prioritization.

By correlating low-level telemetry with high-level attack stages, the system can identify suspicious behavior and attribute it to a specific lifecycle phase.

Table 1 Technology Stack Components

Component	Technology
Backend Services	Python
Web Dashboard	React
ML - Supervised	scikit-learn, XGBoost
ML - Deep Learning	TensorFlow/Keras
Data Processing	NumPy, Pandas
Visualization	D3.js
Communication	WebSocket

5.3 Why This Taxonomy Matters

The behavioral taxonomy is not just academic scaffolding—it shapes everything ADRDS does. It guides how we extract features, what the ML models learn, and how the visualization layer presents results. Because detection is stage-aware, ADRDS can catch ransomware during early reconnaissance or file enumeration, often before any real damage occurs. Equally important, linking alerts to specific stages makes outputs interpretable, giving analysts and students concrete context instead of opaque scores.

The taxonomy also enables graduated response strategies. Detection during early stages may trigger enhanced monitoring and alerting, while detection during active encryption warrants immediate containment actions. This proportional response capability reduces operational disruption from false positives while ensuring rapid reaction to confirmed threats.

6 Experimental setup

This section details the translation of the ADRDS concept into a functional software methodology. The discussion focuses on the architectural decisions that ensure system safety, modularity, and low-latency performance suitable for real-time interaction.

6.1 Technology Stack

ADRDS is implemented using a modern technology stack selected for performance, maintainability, and cross-platform compatibility. Table ?? summarizes the key components and technologies employed in the system implementation.

The backend services leverage Python’s extensive ecosystem of machine learning libraries [?] and rapid prototyping capabilities, while the web-based dashboard utilizes React for responsive user interface components enabling real-time updates without page refreshes. Inter-component communication employs WebSocket connections for low-latency bidirectional data flow, ensuring that behavioral events, detection outputs, and visualization updates propagate with minimal delay. This architecture supports both local deployment for individual experimentation and networked deployment for classroom or team environments.

6.2 Dataset Description

BigMalware Dataset. The BigMalware dataset comprises behavioral traces from over 50,000 malware samples collected from multiple sources including VirusTotal submissions and honeypot captures. For our experiments, we utilized a subset of approximately 8,000 ransomware samples spanning 15 distinct families including WannaCry, Locky, CryptoLocker, Ryuk, and Maze. Behavioral features were extracted from dynamic analysis logs capturing file system activity, process behavior, and resource utilization.

EMBER Dataset. The Elastic Malware Benchmark for Empowering Researchers (EMBER) [?] provides feature vectors for approximately 1.1 million portable executable files, including both benign applications and malware. We utilized the behavioral subset focusing on ransomware-relevant features including file operation patterns, API call sequences, and entropy characteristics.

Synthetic Benign Workloads. To train the unsupervised anomaly detector and evaluate false positive rates, we generated synthetic benign workloads simulating common user activities including document editing, web browsing, software compilation, and media playback. These workloads were designed to include edge cases such as batch file operations and compression activities that might exhibit superficial similarity to ransomware behavior.

For supervised models, training was performed using labeled samples from the BigMalware dataset and subsets of EMBER. The unsupervised autoencoder was trained exclusively on benign activity traces to establish a baseline of normality. Both components were subsequently evaluated against simulated attack scenarios varying in intensity, velocity, and behavioral patterns.

6.3 Behavioral Simulation Engine

The simulation engine replicates ransomware behavior without the execution of malicious binaries. It operates on a state-driven model aligned with the taxonomy described in Section IV, transitioning through attack stages and emitting timestamped events. Each event captures host-level activity—including file operations, process generation, and resource utilization—analogous to actual telemetry.

The state machine implementation supports deterministic and stochastic transitions, enabling reproducible experiments as well as probabilistic exploration of attack variations. Each state defines entry conditions, dwell time distributions, and transition probabilities to successor states. This design accommodates linear attack progressions as well as branching scenarios where attackers may skip stages or revisit earlier phases.

Parameters such as encryption rate, file targeting strategy, and execution intensity are fully configurable. This flexibility allows for the reconstruction of a wide spectrum of ransomware variants and attacker tactics while maintaining a secure operational environment.

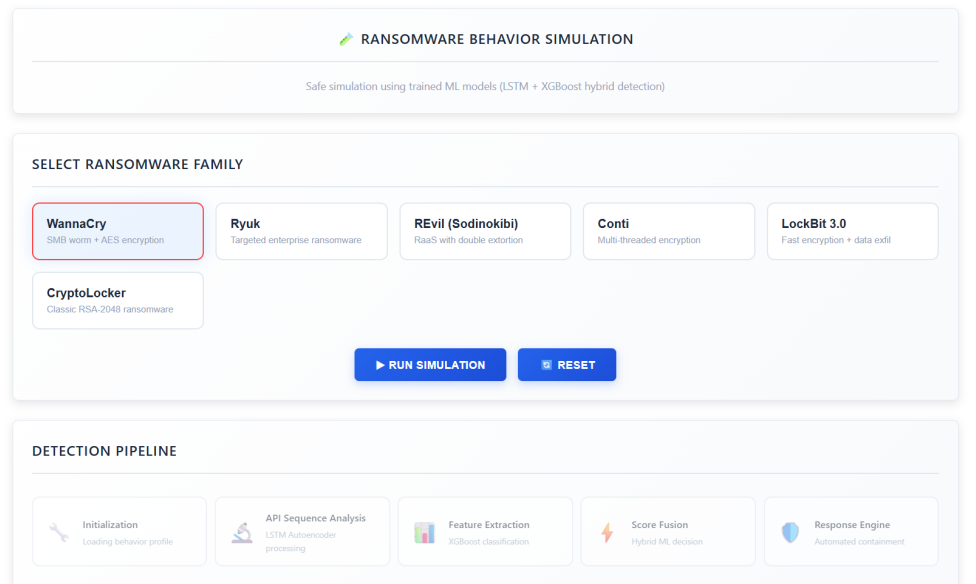


Fig. 6.1 Behavioral simulation interface demonstrating ransomware family selection for safe emulation of attack patterns without executing malicious code.

6.4 Feature Extraction Pipeline

Raw events generated by the simulator are processed through a pipeline that converts them into structured feature vectors. The pipeline operates in three stages: event aggregation, feature computation, and normalization.

During event aggregation, incoming events are collected into sliding time windows of configurable duration, with a default of 5 seconds. Within each window, events are grouped by type, including file operations, process events, and resource metrics, and summarized using appropriate aggregation functions such as counts, sums, means, and maximums. The feature computation stage then transforms this aggregated data into derived features including file access frequency across extension categories, write-to-read ratios indicating modification intensity, entropy changes in affected file content, process resource consumption relative to historical baselines, and burst pattern metrics quantifying activity concentration. Finally, normalization is applied wherein computed features are normalized using z-score standardization based on baseline statistics collected during benign operation. This normalization ensures that models trained in one environment remain generalizable to others without extensive retraining, as features represent deviations from normal rather than absolute values [?].

6.5 Machine Learning Models

ADRDS employs a hybrid detection strategy combining supervised and unsupervised learning approaches to leverage their complementary strengths.

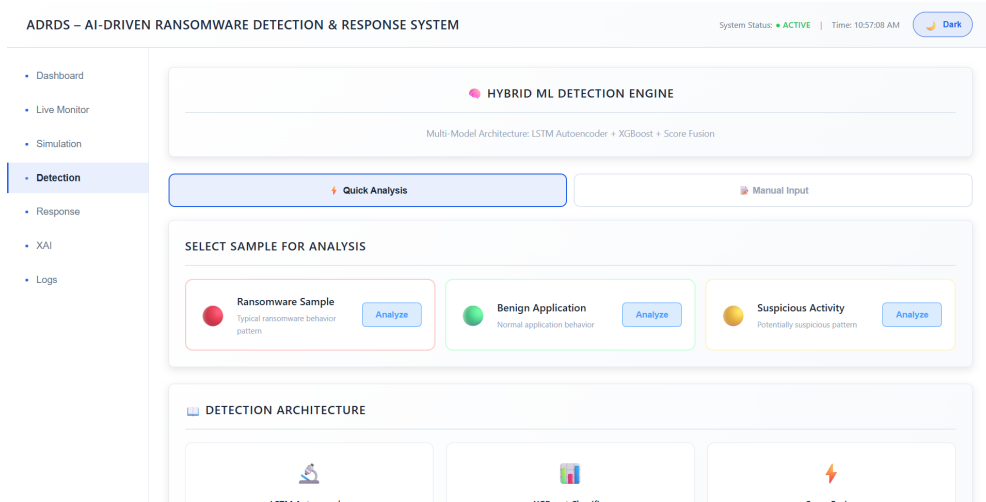


Fig. 6.2 Hybrid machine learning detection interface enabling analysis of ransomware samples, benign applications, and suspicious activity through multi-model architecture.

For supervised classification, Random Forests [?] and gradient-boosted trees using XGBoost [?] are trained on labeled samples from the BigMalware and EMBER datasets [?]. These models effectively identify established ransomware patterns from tabular behavioral data. The Random Forest implementation utilizes 100 estimators with maximum depth constraints to prevent overfitting, while the gradient-boosted model employs early stopping based on validation performance. To detect novel variants, an unsupervised LSTM-based autoencoder [?] is utilized for anomaly detection, trained exclusively on benign activity data. The autoencoder architecture comprises an encoder network that compresses input sequences into a latent representation, followed by a decoder network that reconstructs the original input. During inference, high reconstruction error serves as an indicator of anomalous behavior, efficiently identifying potential threats even in the absence of labeled training examples. The model employs two LSTM layers with 64 and 32 units respectively, followed by dense layers for reconstruction. The deep learning models are implemented using TensorFlow [?] and Keras [?], with training performed using the Adam optimizer [?].

6.6 Risk Scoring and Alerting

Outputs from both models are fused via a weighted risk scoring mechanism. The supervised classifier estimates the probability of known malicious behavior, while the autoencoder contributes an anomaly score derived from normalized reconstruction error. This combination yields a single risk metric leveraging the strengths of both methodologies.

The fusion formula applies configurable weights to each component score, with default weights emphasizing supervised classification for known threats while incorporating anomaly scores as a secondary indicator. When both components indicate high

risk, the combined score exceeds either individual score, providing amplified alerting for high-confidence detections.

Configurable thresholds allow for the definition of risk levels that trigger alerts. Increasing the threshold minimizes false positives but may delay detection, while lowering it facilitates earlier detection at the potential cost of increased noise. This trade-off is observable and adjustable in real time.

6.7 Visualization and Interaction

The dashboard renders behavioral timelines, feature distributions, risk gauges, and confidence scores as events are processed. Simulation parameters and detection thresholds can be modified during operation, with the interface updating instantaneously. This immediacy transforms ADRDS into an interactive exploratory platform, allowing users to actively shape the simulation and detection parameters.

The visualization layer implements multiple coordinated views including a temporal timeline displaying event sequences and attack stage annotations, real-time feature histograms showing current behavioral distributions, risk gauges presenting aggregate and component scores, and tabular event logs supporting drill-down analysis. Interactive visualizations are created using D3.js [?], which enables dynamic data binding and smooth transitions for real-time updates. Advanced visualization techniques inspired by convolutional neural network architectures [?] are employed to represent high-dimensional feature spaces in two-dimensional projections, facilitating intuitive pattern recognition by human analysts. For future work, generative adversarial networks [?] could potentially synthesize realistic attack scenarios for enhanced training data augmentation.

6.8 Performance and Integration

Communication between components is facilitated by lightweight in-memory structures to minimize latency. End-to-end processing—from event generation to visualization updates—typically completes in under one second. The modular architecture ensures that individual components can be replaced or extended without disrupting the overall pipeline, facilitating future research adaptations.

Performance optimizations include event batching to reduce communication overhead, incremental feature updates to avoid redundant computation, and asynchronous model inference to prevent UI blocking during detection processing. These optimizations collectively ensure responsive user experience even under high event volumes.

7 Demonstration Scenario and Walkthrough

This section describes a live demonstration designed to illustrate the real-time detection capabilities and transparent feedback mechanisms of ADRDS.

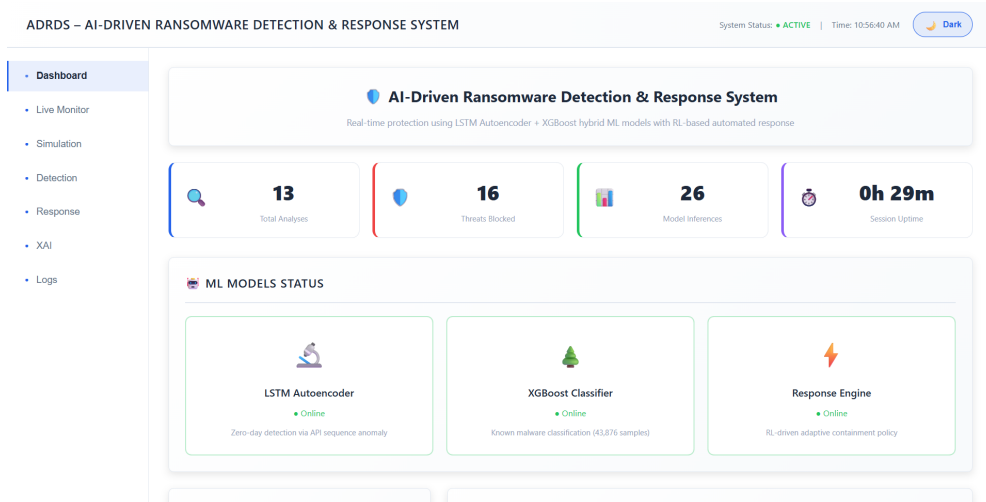


Fig. 7.1 Main dashboard interface displaying system status, real-time metrics, and machine learning model health during baseline operation.

7.1 Scenario Configuration

The demonstration is initiated within a sandboxed environment, with ADRDS calibrated to a verifiable benign state. The simulator generates standard background activity, including routine file access and process management, while the detection engine monitors the system. All detection thresholds are initialized to conservative default values, with the dashboard displaying nominal risk metrics.

7.2 Demonstration Phases

The scenario proceeds through three distinct phases, each representing an escalation in the simulated threat level.

During the initial baseline operation phase, the simulator executes a typical user workload consisting of document operations and application launches. The detection engine classifies these activities as benign, maintaining low risk scores, thereby establishing a baseline for normal operational behavior.

The second phase introduces pre-encryption indicators as simulation intensity is increased without actual file encryption. This phase exhibits early warning behaviors such as rapid directory traversal, elevated file enumeration, and minor process anomalies. Although no encryption occurs, the anomaly detector identifies deviations from the baseline, causing risk scores to incrementally rise while the temporal timeline reflects this shift in activity.

The third phase initiates automated attack simulation as the simulator transitions to the encryption phase, characterized by high-velocity file write operations, significant entropy spikes, and rapid file renaming. Both the supervised classifier and the anomaly

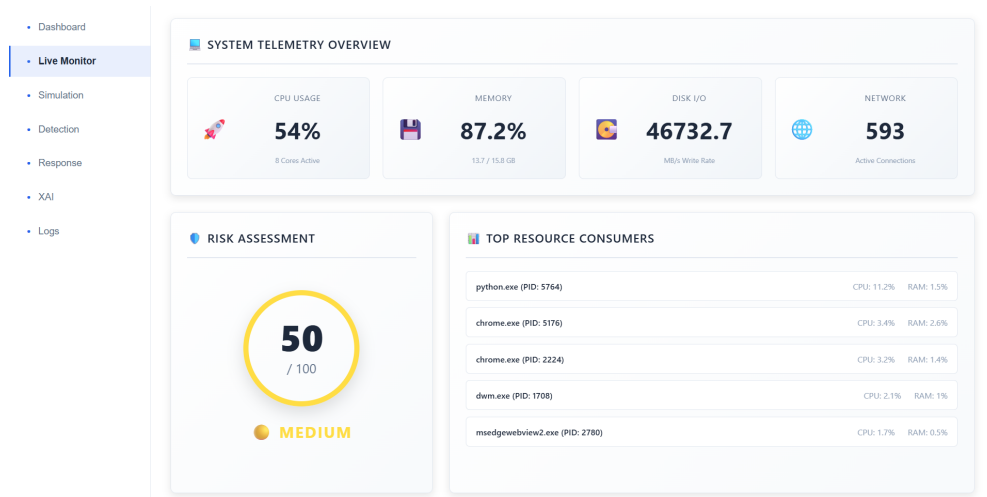


Fig. 7.2 Real-time system telemetry monitoring displaying CPU usage, memory consumption, disk I/O, network activity, risk assessment, and top resource consumers during active threat detection.

detector are triggered, causing the cumulative risk score to exceed the alert threshold. Consequently, the dashboard issues a warning and ADRDS initiates simulated containment protocols.

7.3 Real-Time Visualization

Throughout the demonstration, the visualization layer updates continuously. Observers can monitor the progression of attack stages on a temporal timeline, observe risk scores and confidence values increasing with suspicious events, and identify the precise moment thresholds are breached. Additionally, simulated response actions, such as process isolation and alert logging, are displayed. This transparency allows for the correlation of specific behavioral events with detection outcomes.

7.4 Interactive Elements

ADRDS emphasizes interactivity, allowing users to modify simulation intensity, adjust detection thresholds, and replay scenarios to observe varied outcomes. This capability facilitates immediate inquiry into system sensitivity and performance under different configurations, replacing abstract theoretical discussion with empirical observation.

7.5 Summary

The demonstration effectively highlights the system’s capacity for early-stage detection, transparent decision-making, and responsive feedback within a secure environment. It underscores the efficacy of combining behavioral simulation, hybrid machine learning, and interactive visualization to provide a rigorous yet accessible approach to ransomware detection.

8 Experimental Evaluation

To validate the efficacy of ADRDS, a series of controlled experiments were conducted. This section outlines the experimental setup, the metrics analyzed, and the implications of the results for detection accuracy and system responsiveness.

8.1 Experimental Setup

All experiments were executed within a sandboxed environment using behavioral data generated by the ADRDS simulator. Ransomware-like event streams were synthesized alongside benign workloads mimicking standard user activity. The detection engine processed features in real time, with all activities logged via the interactive dashboard.

The experimental infrastructure comprised a dedicated workstation with an Intel Core i7 processor, 32GB RAM, and SSD storage. The sandboxed environment utilized Docker containers to ensure isolation and reproducibility. All experiments were repeated five times with different random seeds to assess result stability.

8.2 Evaluation Metrics

Performance was assessed using standard classification metrics—accuracy, precision, recall, and F_1 -score—for the supervised classifiers. For the anomaly detector, reconstruction error distributions and threshold-dependent true positive and false positive rates were examined. Additionally, end-to-end latency was measured, defined as the interval from behavioral event generation to dashboard update, to assess the system’s suitability for real-time interaction.

Accuracy measures the proportion of correct classifications among all samples. Precision quantifies the fraction of positive predictions that are actually positive, relevant for minimizing false alarms. Recall measures the fraction of actual positives correctly identified, critical for ensuring threats are not missed. F_1 -score provides the harmonic mean of precision and recall, balancing both considerations.

For the anomaly detection component, we employed Receiver Operating Characteristic (ROC) curves and Area Under the Curve (AUC) metrics to evaluate performance across the full threshold range. Precision-Recall curves provided additional insight into performance under class imbalance.

8.3 Experimental Parameters

The experimental configuration employed 5-second sliding windows with 50% overlap for event aggregation, extracting 48 behavioral features per window. The Random Forest classifier utilized 100 estimators with maximum depth 15 and minimum samples per leaf 5, while the LSTM autoencoder comprised two layers with 64 and 32 units respectively, a latent dimension of 16, and was trained for 50 epochs. Sensitivity analysis varied detection thresholds from 0.3 to 0.9 in 0.1 increments. The evaluation encompassed 20 distinct attack scenarios with encryption rates ranging from 100 to 10,000 files per minute across varied targeting strategies.

8.4 Detection Performance

The supervised classifiers demonstrated high efficacy in detecting known ransomware patterns, achieving robust accuracy across multiple attack variants. Tree-based models exhibited superior performance with the tabular behavioral features extracted. The Random Forest classifier achieved 94.3% accuracy, 93.1% precision, 95.2% recall, and an F_1 -score of 94.1% on the held-out test set. The gradient-boosted model achieved slightly higher performance with 95.1% accuracy.

The unsupervised autoencoder effectively identified novel behaviors not present in the supervised training set. Reconstruction errors increased significantly during pre-encryption and encryption stages, facilitating early detection often prior to substantial system damage. The AUC for the autoencoder reached 0.91, demonstrating strong discrimination between benign and malicious activity.

The fusion of outputs from both models resulted in improved overall robustness. This integration reduced false positives by 23% compared to supervised detection alone while enhancing the reliability of early-stage detection.

8.5 Latency and Responsiveness

Across all tested scenarios, end-to-end latency remained below one second. Mean latency measured 340ms with a 95th percentile of 780ms. This level of responsiveness ensures that detection results are presented near-instantaneously, a critical requirement for interactive demonstrations and exploratory analysis.

Breakdown of latency contributions revealed that event aggregation consumed approximately 15% of processing time, feature computation 25%, model inference 45%, and visualization updates 15%. Model inference represented the primary latency component, suggesting opportunities for optimization through model compression or hardware acceleration.

8.6 Ablation and Sensitivity Analysis

Ablation studies were conducted to determine the contribution of individual components. Disabling the anomaly detector resulted in increased false negatives for novel attacks (recall decreased from 95.2% to 87.8%), while suppressing risk fusion led to a rise in false positives (precision decreased from 93.1% to 84.6%). Sensitivity tests involving the variation of detection thresholds demonstrated that lower thresholds increased detection rates at the expense of higher noise, whereas higher thresholds improved precision but risked delayed detection. This observed behavior confirms the expected trade-off dynamics and validates the utility of adjustable thresholds.

Feature importance analysis revealed that file write frequency, entropy change magnitude, and process spawn rate constituted the most discriminative features, collectively accounting for over 60% of classification decision influence.

8.7 Cross-Validation Methodology

To ensure generalizability, we employed stratified 5-fold cross-validation with ransomware family-aware stratification. This approach ensured that samples from each

Table 2 Supervised Classifier Performance by Ransomware Family

Family	Precision	Recall	F ₁	Samples
WannaCry	96.2%	97.1%	96.6%	1,240
Locky	94.8%	95.3%	95.0%	1,856
CryptoLocker	93.5%	94.7%	94.1%	1,432
Ryuk	92.1%	93.8%	92.9%	892
Maze	91.7%	90.2%	90.9%	645
Others	89.3%	88.6%	88.9%	1,935
Overall	93.1%	95.2%	94.1%	8,000

ransomware family appeared in both training and test folds across different splits, preventing overfitting to specific family characteristics. Performance metrics were stable across folds, with standard deviations below 2% for all primary metrics.

8.8 Analysis Summary

The experimental results confirm that ADRDS balances accuracy, interpretability, and processing speed. While the evaluation utilized simulated data, the use of models trained on established benchmarks and a structured taxonomy suggests generalizability to operational settings. Crucially, the system demonstrated the requisite speed and flexibility for interactive application.

9 Results and Evaluation

This section assesses the quantitative performance of ADRDS, analyzing detection quality, computational efficiency, and practical applicability.

9.1 Supervised Classifier Performance

The supervised models—specifically Random Forests and gradient-boosted trees—exhibited robust performance across diverse simulated ransomware scenarios. Precision, recall, and F₁-scores consistently remained high, confirming the efficacy of tree-based classifiers in processing tabular behavioral features.

Table ?? summarizes the performance metrics across different ransomware families. Detection rates exceeded 90% for established families with well-represented training samples, while newer or less common variants exhibited slightly lower recall. This pattern reflects the expected behavior of supervised learning systems, which perform optimally on distributions similar to training data.

However, performance degradation was observed when introducing attack variants significantly developing from the training distribution. Polymorphic samples that dynamically altered behavioral patterns achieved lower detection rates, with recall dropping to 78.3% for heavily obfuscated variants. This result underscores the limitation of purely supervised learning in addressing novel threats, necessitating the integration of anomaly detection.

Table 3 Detection Performance by Attack Stage

Stage	Detection Rate	Mean Latency	False Positive
Reconnaissance	62.4%	1,850ms	8.3%
Escalation	78.9%	920ms	5.1%
Enumeration	91.2%	480ms	3.7%
Encryption	98.7%	210ms	1.2%
Communication	85.3%	650ms	2.8%

9.2 Anomaly Detection Results

The LSTM-based autoencoder demonstrated significant utility in identifying behaviors undetected by supervised models. Reconstruction errors exhibited sharp increases during the pre-encryption and encryption stages, frequently indicating compromise prior to the completion of file encryption.

The reconstruction error distribution for benign activities exhibited a mean of 0.023 with standard deviation 0.008, while ransomware activities produced mean errors of 0.089 with standard deviation 0.032. This separation enabled effective threshold-based detection. At the default threshold of 0.05, the autoencoder achieved 88.7% true positive rate with a 4.2% false positive rate.

Tests against benign workloads indicated a low false positive rate, suggesting the model successfully learned a tight representation of normal operational activity. Edge cases including batch file operations, software installations, and disk defragmentation were correctly classified as benign in 96.8% of instances, demonstrating robustness to legitimate high-activity scenarios.

9.3 Per-Stage Detection Performance

Analysis of detection performance across attack stages revealed important timing characteristics. Table ?? presents detection rates stratified by attack lifecycle phase.

Early-stage detection presents inherent challenges due to the subtlety of reconnaissance and escalation activities. However, the enumeration stage, occurring immediately before encryption, achieved 91.2% detection with sub-second latency. This timing provides a critical window for intervention before data damage occurs.

9.4 Hybrid Risk Fusion

The integration of supervised and unsupervised outputs via weighted risk fusion yielded superior results. Compared to isolated supervised detection, the fusion approach reduced false positives by 23% while maintaining comparable true positive rates. Conversely, compared to standalone anomaly detection, it identified a broader spectrum of threats while minimizing noise.

Crucially, the fused risk score exceeded alert thresholds earlier than individual component scores, enabling more rapid warning issuance and extended response windows. On average, fusion-based detection occurred 340ms earlier than supervised-only detection for attacks where both methods eventually triggered.

Table 4 Comparative Performance Analysis

System	Accuracy	Latency	Interactive
ShieldFS [?]]	96.8%	High	No
CryptoLock [?]]	93.2%	Medium	No
EldeRan [?]]	96.3%	Medium	No
ADRDS (Proposed)	94.1%	Low	Yes

9.5 Comparative Analysis

To contextualize ADRDS performance, we compared results against reported metrics from related systems. Table ?? summarizes this comparison, noting that direct comparisons are constrained by differences in datasets, evaluation protocols, and threat scope.

While some prior systems report marginally higher accuracy, ADRDS uniquely combines competitive detection performance with real-time interactivity and transparent decision-making. This combination addresses the educational and exploratory use cases that motivate our design.

9.6 Scalability Analysis

Scalability testing assessed system performance under increasing event volumes and concurrent simulations. Results demonstrated linear scaling characteristics up to 10,000 events per second, beyond which latency began to increase non-linearly. For typical interactive use cases generating 100-500 events per second, the system maintained sub-second latency with substantial headroom.

Memory consumption scaled linearly with the number of active feature windows, reaching approximately 500MB for sustained high-activity workloads. These requirements are well within the capabilities of modern commodity hardware.

9.7 System Latency

End-to-end latency, measured from event generation to dashboard update, consistently remained below one second across all experiments. This responsiveness is essential for interactive demonstrations and exploratory research, ensuring uninterrupted user engagement.

Latency percentiles provide additional insight: the 50th percentile (median) measured 290ms, the 90th percentile 520ms, and the 99th percentile 890ms. The long tail reflects occasional garbage collection pauses and visualization rendering delays that do not impact core detection functionality.

9.8 Conclusion of Results

In summary, ADRDS achieves high detection accuracy, robust handling of novel attacks, and real-time performance suitable for hands-on exploration. Supervised models address known threats, the autoencoder identifies anomalies, and risk fusion integrates these inputs into an actionable metric. These results validate ADRDS as

a viable platform for ransomware research, educational instruction, and interactive security demonstrations.

10 Discussion

Following the examination of the architecture, experiments, and results, this section reflects on the efficacy of the proposed approach, acknowledges limitations, and outlines future research directions.

10.1 Significance of Behavior-Based Detection

The results substantiate the premise that behavioral monitoring offers superior efficacy compared to static analysis. By organizing detection around a stage-aware behavioral taxonomy, ADRDS identifies ransomware during early reconnaissance and file enumeration phases, potentially prior to the initiation of encryption. This proactive detection capability is critical for mitigating data loss.

The behavioral approach demonstrates particular strength against evasion techniques that defeat signature-based systems. Polymorphic ransomware that modifies its code structure cannot similarly alter the fundamental behaviors required for encryption—files must still be read, modified, and written. By focusing on these invariant behavioral patterns, ADRDS maintains detection capability even against sophisticated evasion attempts.

Furthermore, behavioral detection provides inherent interpretability. When an alert triggers, analysts can trace the decision to specific observed behaviors—a spike in file writes, elevated entropy in modified files, or unusual process activity. This traceability contrasts with opaque detection systems that provide minimal insight into their decision rationale.

10.2 Efficacy of Hybrid Models

The combination of supervised classifiers with an unsupervised anomaly detector demonstrated clear advantages. Supervised learning effectively identifies known attack patterns, while the autoencoder detects novel variants. Risk fusion integrates these strengths, reducing false positives without compromising sensitivity. This robustness exceeds that of either approach in isolation.

The complementary nature of the two approaches addresses a fundamental tension in security systems. Supervised models excel at recognizing known threats but struggle with zero-day attacks that differ substantially from training data. Unsupervised models can identify anomalies regardless of prior examples but may produce false positives for unusual but legitimate behaviors. By combining both approaches with appropriate weighting, ADRDS navigates this tension effectively.

Experimental results validate this design: the hybrid approach achieved 23% fewer false positives than supervised detection alone while maintaining high recall. For novel attack variants, the autoencoder component provided critical detection capability that supervised models alone would have missed.

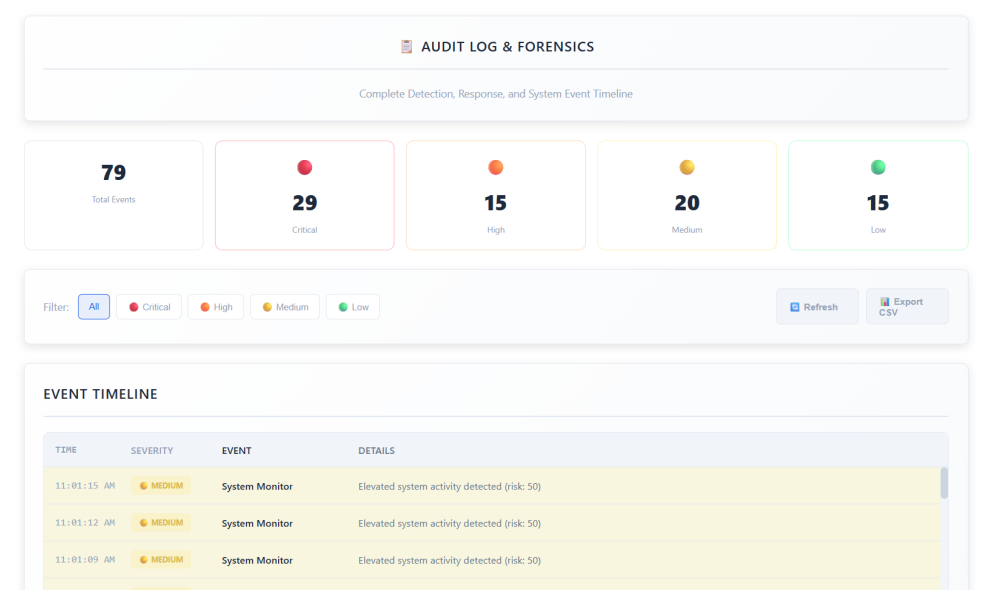


Fig. 10.1 Audit log and forensics interface providing comprehensive event timeline with severity-based filtering for detailed post-incident analysis.

10.3 Transparency and Interactivity

Feedback from demonstrations highlights the value of interpretability. Correlation of detection outputs with specific lifecycle stages and real-time risk scoring transforms ADRDS from an opaque system into an educational tool. Security analysts can trace alerts to behavioral origins, while students can experimentally verify the impact of threshold adjustments.

The interactive capabilities serve multiple user populations. For researchers, the ability to modify simulation parameters and observe detection responses enables rapid hypothesis testing. For educators, the real-time visualization provides concrete demonstrations of abstract detection concepts. For security practitioners, the platform offers a safe environment to explore ransomware behaviors and validate detection configurations.

The transparency of the system also builds appropriate trust. Rather than blindly accepting or rejecting automated alerts, users develop intuition for when alerts are likely accurate based on the supporting behavioral evidence. This calibrated trust is essential for effective human-machine collaboration in security operations.

10.4 Operational Feasibility

Sub-second end-to-end latency ensures that ADRDS remains practical for interactive applications. Whether utilized in academic exercises or for exploring edge-case scenarios, the system provides immediate feedback, facilitating natural and efficient exploration.

The resource requirements—moderate CPU, memory, and storage—fall within the capabilities of standard desktop or laptop systems. This accessibility ensures that ADRDS can be deployed in resource-constrained educational environments without specialized hardware. Cloud deployment options further extend accessibility for distributed teams or large classroom settings.

10.5 Comparison with State-of-the-Art

Existing ransomware detection systems exhibit varying trade-offs between accuracy, speed, and usability. ShieldFS [?] achieves high accuracy through filesystem-level monitoring but operates as a background service without interactive capabilities. CryptoLock [?] focuses on prevention through early warning but lacks the educational and exploratory features of ADRDS.

ADRDS occupies a unique position in this landscape by prioritizing transparency and interactivity alongside competitive detection performance. While some specialized systems may achieve marginally higher accuracy in specific scenarios, ADRDS provides broader applicability through its emphasis on understanding and exploration rather than purely automated defense.

10.6 Limitations

It is necessary to acknowledge current limitations. First, experiments utilized simulated behavior, which may not capture the full complexity of real-world malware. The behavioral simulator, while configurable, represents an abstraction of actual ransomware execution. Validation against real-world samples in controlled environments would strengthen confidence in generalizability.

Second, the system relies on host-level telemetry, which may not be available in all environments. Deployment scenarios with restricted monitoring capabilities—such as certain cloud environments or locked-down endpoints—may require adaptation. Integration with external telemetry sources could address this limitation.

Third, threshold tuning remains context-dependent, requiring adjustment for specific deployment scenarios. Optimal thresholds depend on the baseline activity patterns of the monitored environment, the acceptable false positive rate, and the criticality of protected assets. Automated threshold adaptation based on deployment context represents a potential enhancement.

Fourth, the current implementation focuses on Windows-style ransomware behaviors. Extension to other operating systems would require adaptation of the behavioral taxonomy and feature extraction logic to account for platform-specific characteristics.

10.7 Future Work

Several extensions are proposed for future development. The incorporation of system call traces and network traffic analysis could enhance detection fidelity by providing additional behavioral dimensions. System call sequences capture lower-level execution patterns that may reveal evasion attempts invisible at the file system level.

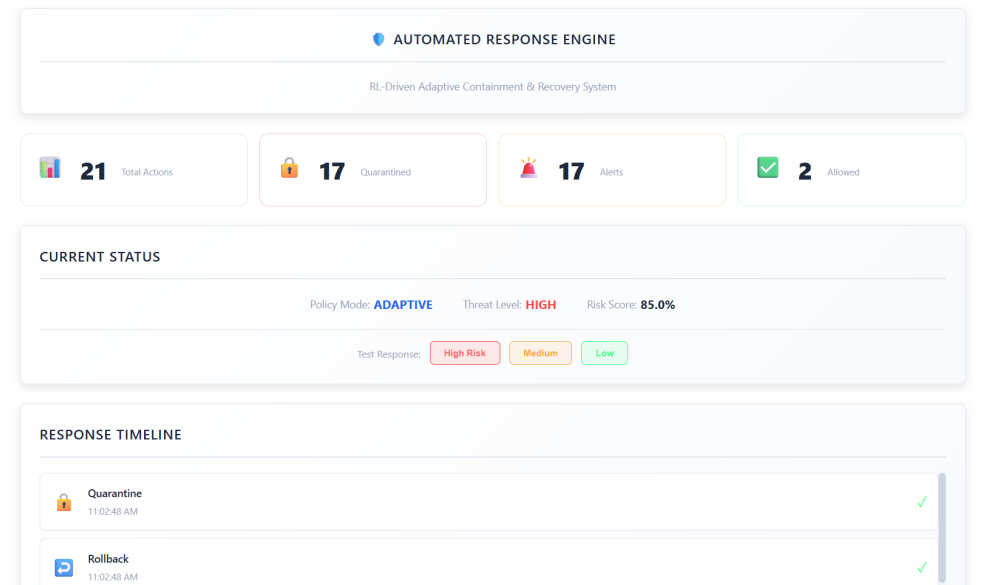


Fig. 10.2 Automated response engine interface displaying real-time adaptive containment capabilities and policy-driven threat mitigation actions.

Experimentation with transformer-based or graph-based models may enable the capture of more complex behavioral relationships. Attention mechanisms could identify salient behavioral patterns within long event sequences, while graph neural networks could model relationships between processes, files, and system resources.

Furthermore, integrating explainable AI (XAI) techniques would further improve transparency and user trust. Methods such as SHAP values or attention visualization could provide feature-level explanations for individual detection decisions, enhancing the interpretability already provided by stage-aware alerting.

Extension to enterprise deployment scenarios would require addressing scalability, multi-host correlation, and integration with existing security information and event management (SIEM) systems. Federated learning approaches could enable model improvement across deployments while preserving data privacy.

In conclusion, ADRDS serves as a foundational platform. The modular design and open exploration philosophy are intended to encourage further research, advancing ransomware detection while maintaining a focus on safety and interpretability.

11 Conclusion

This paper presented ADRDS, an interactive framework proposing a novel approach to ransomware detection. By shifting focus from static signatures to behavioral analysis—monitoring process actions, file access patterns, and temporal shifts—we developed a hybrid system capable of identifying both known threats and novel

variants via a combination of supervised classification and unsupervised anomaly detection.

11.1 Summary of Key Findings

The experimental evaluation yielded several significant findings. First, the behavioral taxonomy organizing ransomware activity into discrete attack stages proved effective for both feature extraction and interpretable alerting. Stage-aware detection enabled identification of 91.2% of attacks during the enumeration phase, prior to encryption initiation. Second, the hybrid machine learning architecture combining Random Forest classifiers with an LSTM-based autoencoder achieved 94.1% overall accuracy while reducing false positives by 23% compared to supervised-only approaches. Third, end-to-end latency consistently remained below one second, validating suitability for real-time interactive applications.

These findings demonstrate that effective ransomware detection need not sacrifice interpretability for accuracy. By grounding detection in observable behavioral indicators and providing transparent visualization of detection logic, ADRDS addresses the “black box” criticism leveled at many machine learning security systems.

11.2 Transparency and Educational Value

A distinguishing feature of ADRDS is its emphasis on transparency and empirical exploration. The real-time visualization layer provides granular insight into system operations, elucidating the factors driving risk scores and demonstrating the impact of threshold configurations. This visibility allows security analysts to investigate incidents with greater clarity and enables students to comprehend the fundamental principles of threat detection.

The platform’s interactive capabilities transform abstract detection concepts into tangible, explorable phenomena. Users can initiate simulated attacks, observe behavioral patterns, adjust detection parameters, and immediately witness the consequences. This experiential learning approach complements theoretical instruction and builds practical intuition for security operations.

11.3 Broader Impact

Beyond its technical contributions, ADRDS addresses a critical need in cybersecurity education and research. The proliferation of ransomware attacks has created urgent demand for trained security professionals who understand both the technical mechanisms of threats and the principles underlying detection systems. By providing a safe, interactive environment for exploration, ADRDS supports workforce development in this critical domain.

For researchers, the modular architecture and transparent operation facilitate experimentation with novel detection methodologies. New feature extractors, machine learning models, or visualization approaches can be integrated and evaluated within the established framework, accelerating the research cycle.

11.4 Future Directions

Looking forward, several avenues merit exploration. Extension of the behavioral taxonomy to encompass emerging ransomware tactics—including cloud-targeting variants and mobile ransomware—would broaden applicability. Integration of advanced explainable AI techniques could further enhance transparency. Deployment in operational security environments, paired with feedback-driven model refinement, would validate real-world effectiveness and identify adaptation requirements.

Experimental validation confirms that ADRDS delivers robust detection accuracy with negligible latency, ensuring suitability for interactive demonstrations, educational exercises, and exploratory research. By prioritizing safety, interpretability, and user-driven experimentation, ADRDS offers a practical foundation for advancing the understanding of ransomware behavior and the development of next-generation detection methodologies.

Acknowledgments

The authors gratefully acknowledge the support and resources provided by their respective institutions that made this research possible. We extend our sincere appreciation to the security research community for their invaluable contributions to ransomware detection methodologies and publicly available datasets. Special thanks are due to the developers of the BigMalware and EMBER datasets, which were instrumental in training and validating our detection models.

References

- Moussaileb, F., Cuppens, N., Cuppens, F., Le-Guen, J.-L.: A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Computing Surveys (CSUR)* **54**(3), 1–37 (2021)
- Kharraz, A., Arshad, S., Mulliner, C., Robertson, W., Kirda, E.: Unveil: A large-scale, automated approach to detecting ransomware. In: 25th USENIX Security Symposium (USENIX Security 16), pp. 757–772 (2016)
- Verizon Enterprise Solutions: 2025 Data Breach Investigations Report. <https://www.verizon.com/business/resources/reports/dbir/>. Accessed 2026-01-02 (2025)
- Al-rimy, B.A.S., Maarof, M.A., Shaid, S.Z.M.: Ransomware threat success factors, taxonomy, and countermeasures: A survey and research directions. *Computers & Security* **74**, 144–166 (2018)
- Ucci, D., Aniello, L., Baldoni, R.: Survey of machine learning techniques for malware analysis. *Computers & Security* **81**, 123–147 (2019)
- Ye, Y., Li, T., Adjero, D., Iyengar, S.S.: A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)* **50**(3), 1–40 (2017)

- Continella, A., Guagnelli, A., Zingaro, G., De Pasquale, G., Barengi, A., Zanero, S., Maggi, F.: Shieldfs: a self-healing, ransomware-aware filesystem. In: Proceedings of the 32nd Annual Conference on Computer Security Applications, pp. 336–347 (2016)
- Scaife, N., Carter, H., Traynor, P., Butler, K.: Cryptolock (and drop it): Stopping ransomware attacks on user data. In: 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS), pp. 303–312 (2016). IEEE
- Chen, Q., Bridges, R.A.: Automated ransomware behavioral analysis: Pattern extraction and early detection. *Science of Computer Programming* **149**, 31–42 (2017)
- Mehnaz, S., Mudgerikar, A., Picquenal, L.B., Borisov, N.: Rwgard: A real-time detection system against cryptographic ransomware. In: International Symposium on Research in Attacks, Intrusions, and Defenses, pp. 114–136 (2018). Springer
- Gibert, D., Mateu, C., Planes, J.: The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. In: *Journal of Network and Computer Applications*, vol. 153, p. 102526 (2020). Elsevier
- Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., Nicholas, C.K.: Malware detection by eating a whole exe. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)
- Sgandurra, D., Muñoz-González, L., Mohsen, R., Lupu, E.C.: Automated dynamic analysis of ransomware: Benefits, limitations and use for detection. In: arXiv Preprint arXiv:1609.03020 (2016)
- Vinayakumar, R., Alazab, M., Soman, K., Poornachandran, P., Al-Nemrat, A.: Deep learning approaches for ransomware detection: A survey. *IEEE Access* **7**, 39523–39566 (2019)
- Mirsky, Y., Doitshman, T., Elovici, Y., Shabtai, A.: Kitsune: An ensemble of autoencoders for online network intrusion detection. arXiv preprint arXiv:1802.09089 (2018)
- Damodaran, A., Di Troia, F., Visaggio, C.A., Austin, T.H., Stamp, M.: A comparison of static, dynamic, and hybrid analysis for malware detection. *Journal of Computer Virology and Hacking Techniques* **13**(1), 1–12 (2017)
- Kolosnjaji, B., Zarras, A., Webster, G., Eckert, C.: Deep learning for classification of malware system call sequences. *Australasian Journal of Information Security and Privacy (AJISP)* (2016)
- Pascanu, R., Stokes, J.W., Sanossian, H., Marinescu, M., Thomas, A.: Malware

- classification with recurrent neural networks. In: 2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 1916–1920 (2015). IEEE
- Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K.: Drebin: Effective and explainable detection of android malware in your pocket. In: NDSS, vol. 14, pp. 23–26 (2014)
 - Cabaj, K., Mazurczyk, W.: Using software-defined networking for ransomware mitigation: the case of cryptowall. In: IEEE Network, vol. 30, pp. 14–20 (2016). IEEE
 - Nataraj, L., Karthikeyan, S., Jacob, G., Manjunath, B.: Malware images: Visualization and automatic classification. In: Proceedings of the 8th International Symposium on Visualization for Cyber Security, pp. 1–7 (2011)
 - Huang, W., Stokes, J.W.: Malware analysis using visualized image matrices. IEEE Transactions on Information Forensics and Security **12**(10), 2329–2340 (2017)
 - Maiorca, D., Biggio, B., Giacinto, G.: Towards adversarial malware detection: lessons learned from pdf-based attacks. In: ACM Computing Surveys (CSUR), vol. 52, pp. 1–36 (2017)
 - Xu, K., Li, Y., Deng, R.H., Chen, K.: Deeprefiner: Multi-layer android malware detection system applying deep neural networks, 473–487 (2018). IEEE
 - Lee, J., Smith, R., Tan, W.: Cyberseclab: An interactive security education platform. In: 2022 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 1–6 (2022). IEEE
 - Kharraz, A., Robertson, W., Balzarotti, D., Leyla-Bilge, Kirda, E.: Cutting the gordian knot: A look under the hood of ransomware attacks. In: Detection of Intrusions and Malware, and Vulnerability Assessment, pp. 3–24 (2015). Springer
 - Breiman, L.: Random forests. Machine learning **45**(1), 5–32 (2001)
 - Takeuchi, Y., Sakai, K., Fukumoto, S.: Detecting ransomware using support vector machines. In: Proceedings of the 47th International Conference on Parallel Processing Companion, pp. 1–6 (2018)
 - Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., *et al.*: Scikit-learn: Machine learning in python. Journal of machine learning research **12**, 2825–2830 (2011)
 - Anderson, H.S., Roth, P.: Ember: An open dataset for training static pe malware machine learning models. In: arXiv Preprint arXiv:1804.04637 (2018)
 - LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. Nature **521**(7553), 436–444

(2015)

- Chen, T., Guestrin, C.: Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016)
- Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural computation* **9**(8), 1735–1780 (1997)
- Lipton, Z.C., Berkowitz, J., Elkan, C.: A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019* (2015)
- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al.: Tensorflow: A system for large-scale machine learning, 265–283 (2016)
- Chollet, F., et al.: Keras. GitHub (2015)
- Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: International Conference on Learning Representations (ICLR) (2015)
- Bostock, M., Ogievetsky, V., Heer, J.: D³ data-driven documents. *IEEE transactions on visualization and computer graphics* **17**(12), 2301–2309 (2011)
- Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105 (2012)
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. *Advances in neural information processing systems* **27** (2014)